Thesis Notebook

2D Orbit with the Tolman VII neutron star density profile and mass function, along with dissipative forces

Setting functions up and choosing variables

```
In[•]:= (*R values in Schwarzschild coordinates*)
        RStar = 8;(*Radius of the Star*)
        M = 1;
```

Exterior Schwarzschild Isotropic coordinates definitions $r(\bar{r}), \bar{r}(r), A(\bar{r}), \alpha(\bar{r})$

$r(\bar{r}), \bar{r}(r)$

```
In[•]:= rbarofrOutside[r_] := 1/2 (-M + r + Sqrt[r^2 - 2 M r]);

In[•]:= rbarStar = rbarofrOutside[RStar];

In[•]:= rofrbarOutside[rbar_] := rbar (1 + M/(2 rbar))^2;
```

$A(\bar{r}), \alpha(\bar{r})$

```
In[•]:= A[rbar_] := (1 + M/(2 rbar))^2;
        α[rbar_] := (1 - M/(2 rbar))/(1 + M/(2 rbar));
```

Tolman VII Isotropic coords definitions

Constants

```
In[•]:= rho0 = (15 M)/(8 Pi RStar^3);

In[•]:= ScriptC = M/RStar;
```

New interior rho

In[∘]:=
```
rhoInside[r_] := rho0 (1 - (r/RStar)^2)
```

In[∘]:=
```
massIntegral = 4 Pi Integrate[rhoInside[r] r^2, {r, 0, RStar}] (*Mass sanity check*)
```

Out[∘]= 1

New mass function

In[∘]:=
```
MassFuncInside[r_] := 4 Pi rho0 RStar^3 (1/3 (r/RStar)^3 - 1/5 (r/RStar)^5)
```

Numerically integrating for the pressure and the phi in α = exp(phi)

Defining the ODEs

In[∘]:=
```
(*TOV differential equation*)
dPdr[r_, Pressure_] := -(rhoInside[r] + Pressure) ((MassFuncInside[r] + 4 Pi Pressure r^3)/(r (r - 2 MassFuncInside[r])));
```

In[∘]:=
```
(*Phi differential equation for metric*)
dPhidrMetric[r_, Pressure_] := ((MassFuncInside[r] + 4 Pi Pressure r^3)/(r (r - 2 MassFuncInside[r])));
```

Numerically integrating these coupled ODEs

In[∘]:=
```
rmin = 10^-16;
```

In[∘]:=
```
eq1 = P'[r] == dPdr[r, P[r]];
eq2 = Phi'[r] == dPhidrMetric[r, P[r]];
BCs = {P[RStar] == 0, Phi[RStar] == 1/2 Log[1 - 2 M/RStar]};
(*Phi[RStar] from the Schwarzschild metric*)
```

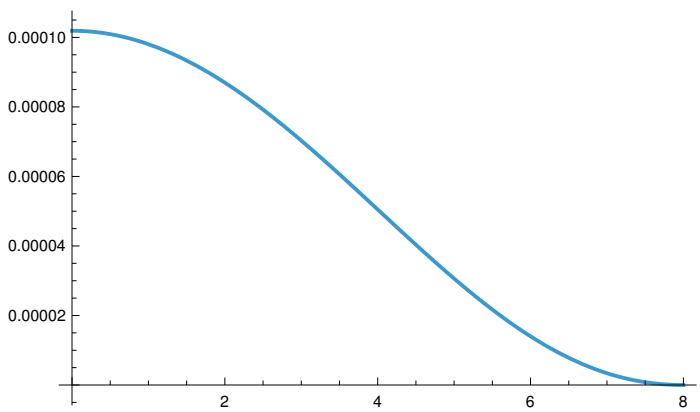
In[∘]:=
```
sol1 = NDSolve[
    {eq1,
     eq2,
     BCs},
    {P, Phi},
    {r, RStar, rmin},
    WorkingPrecision → 55
   ][[1]]; (*Very small rmin to avoid singularity sarp mentioned*)
```

In[∘]:=
```
Pressure[r_] := P[r] /. sol1

PhiMetric[r_] = Phi[r] /. sol1;
```

In[∘]:=
```
Plot[Pressure[r], {r, 0, RStar}, PlotRange → All]
```

Out[∘]=



Solving for $\bar{r}(r)$ and $r(\bar{r})$ inside

In[∘]:=

Defining the ODE

In[∘]:=
```
drbardr[rbar_, r_] := rbar/r (1 - 2 MassFuncInside[r]/r)^(-1/2)
```

Numerically Integrating the above equation

In[∘]:=
```
eq = rbarofr'[r] == drbardr[rbarofr[r], r];
BC = {rbarofr[RStar] == rbarStar};
```

In[∘]:=
```
sol2 = NDSolve[
    {eq,
     BC},
    {rbarofr},
    {r, RStar, rmin},
    WorkingPrecision → 55
   ][[1]];
```

In[∘]:=
```
rbarofrInside[r_] = rbarofr[r] /. sol2;
```

Some sanity checks

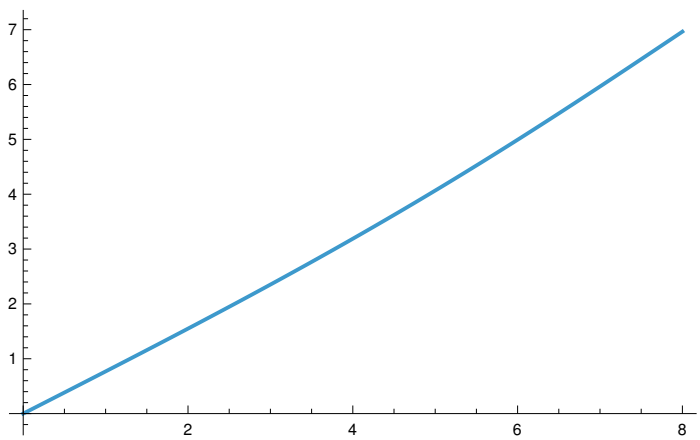
In[∘]:=
```
rbarofrInside[RStar]
rbarofrOutside[RStar] // N
```

Out[∘]= 6.964101615137754587054892683011744733885610507620761 26

Out[∘]= 6.9641

In[∘]:=
```
Plot[rbarofrInside[r], {r, 0, RStar}]
```

Out[∘]=

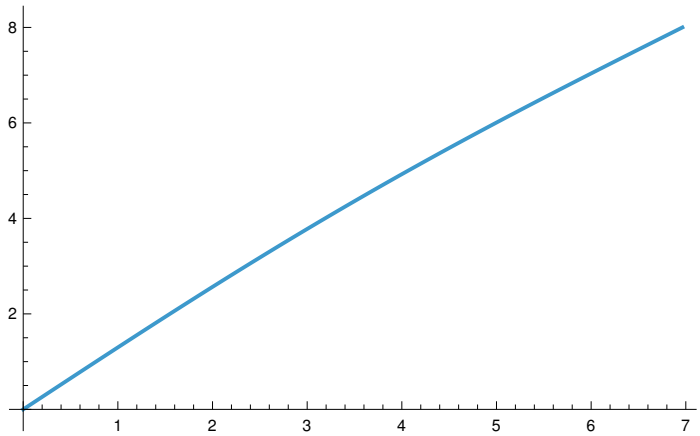


Now using the interpolation trick to obtain $r(\bar{r})$: (causing issues?)

In[∘]:=
```
(*Table[ expression, {variable, start, end, step}*)
data = Prepend[Table[
    {rbarofrInside[r], r},
    {r, rmin, RStar, (RStar - rmin)/10000}] (*10000 points and A is not messy*)
   ], {0, 0}];
```

In[∘]:=
```
rofrbarInside = Interpolation[data];
```

In[∘]:=
```
Plot[rofrbarInside[rbar], {rbar, 0, rbarStar}]
```

Out[∘]=


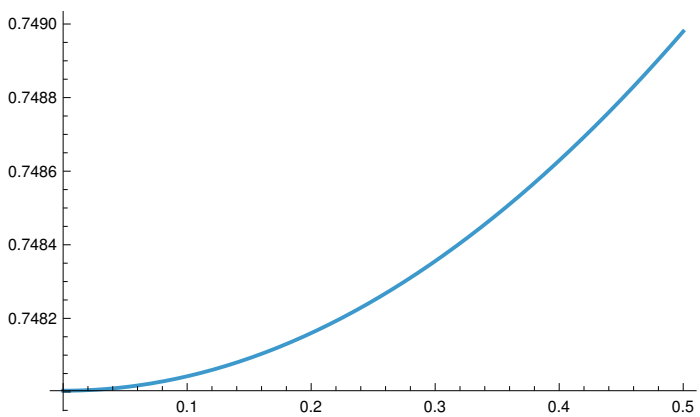
Finding the interior metric functions

Getting αInside($\bar{r}$):

In[∘]:=
```
αInside[rbar_] := Exp[PhiMetric[rofrbarInside[rbar]]]
```

Sanity checks

In[∘]:=
```
Plot[αInside[rbar], {rbar, 0, .5}]
```

Out[∘]=


In[∘]:=
```
αInside[rbarStar]
α[rbarStar] // N
```

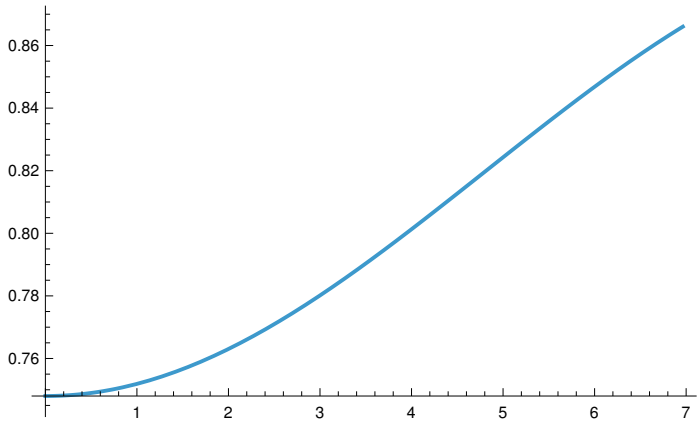
Out[∘]= 0.866025403784438646763723170752936183471402626905190314 0

Out[∘]= 0.866025

In[∘]:=
```
αFull[rbar_] := Piecewise[
  {
   {αInside[rbar], rbar < rbarStar},
   {α[rbar], rbar ≥ rbarStar}
  }
 ]
```

In[∘]:=
```
Plot[αFull[rbar], {rbar, 0, rbarStar}]
```

Out[∘]=



Making the derivative of alpha continuous

Interior derivative in terms of r

Doing things in r first as this is more analytic

In[∘]:=
```
dPhidrMetric[r_] := (MassFuncInside[r] + 4 Pi Pressure[r] r^3 / (r (r - 2 MassFuncInside[r])))
```

In[∘]:=
```
alpha[r_] := Exp[PhiMetric[r]]
DERalphaIns[r_] :=
 dPhidrMetric[r] × alpha[r] (*Something analytic times a numerical integral*)
```

Exterior derivative in terms of r

In[∘]:=
```
DERαOutsiderbardummy = Derivative[1][α][rbar];
```

$$\text{DER}\alpha\text{Outsiderbar}[r_] := \left(\frac{1-\frac{1}{2\,\text{rbar}}}{2\left(1+\frac{1}{2\,\text{rbar}}\right)^2\text{rbar}^2}+\frac{1}{2\left(1+\frac{1}{2\,\text{rbar}}\right)\text{rbar}^2}\right) /.\ \text{rbar} \to \text{rbarofrOutside}[r]$$

```
(*More accurate subbing in after for rbar maybe?*)
```

$$\text{drbardr}[r_] := \frac{\text{rbarofrOutside}[r]}{r}\left(1-\frac{2\,M}{r}\right)^{-\frac{1}{2}}$$

In[∘]:=
```
DERalpha[r_] := drbardr[r] × DERαOutsiderbar[r] (*Chain rule*)
```

Sanity check at boundary

In[∘]:=
```
DERalpha[RStar] // N
DERalphaIns[RStar] // N
```

Out[∘]= 0.0180422

Out[∘]= 0.0180422

In[∘]:=
```
DERalphaIns[rofrbarInside[rbarStar]]
DERalpha[rofrbarInside[rbarStar]]
```

Out[∘]= 0.018042195912175805140910899390686170488987554727191464 9

Out[∘]= 0.018042195912175805140910899390686170488987554727191464 9

In[∘]:=
```
diff = Abs[1 - DERalphaIns[RStar] / DERalpha[RStar]]
```

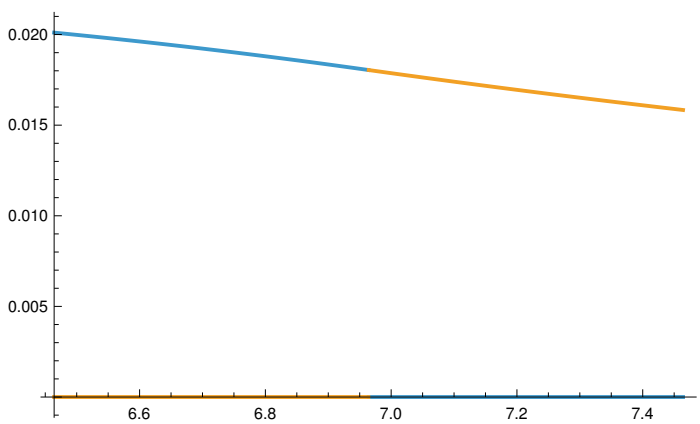
Out[∘]= $0. \times 10^{-56}$

In[∘]:=
```
Abs[Log10[diff]] (*Amount of digits in agreement*)
```

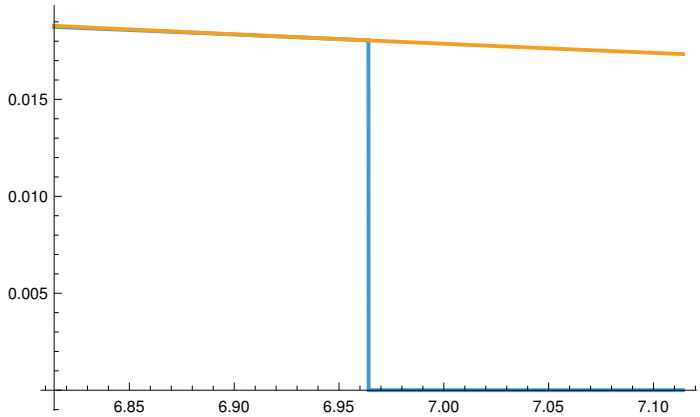
Out[∘]= Indeterminate

In[∘]:=
```
Plot[{Piecewise[{{DERalphaIns[rofrbarInside[rbar]], rbar ≤ rbarStar}}],
  Piecewise[{{DERalpha[rofrbarOutside[rbar]], rbar ≥ rbarStar}}]},
 {rbar, rbarStar - 0.5, rbarStar + 0.5}]
```

Out[∘]=



In[∘]:=
```
Plot[{DERalphaIns[rofrbarInside[rbar]], DERalpha[rofrbarOutside[rbar]]},
 {rbar, rbarStar - 0.15, rbarStar + 0.15}]
```

Out[∘]=


Isotropic coordinates DERα definitions

In[∘]:=
```
DERαOutside[rbar_] := Module[{r = rofrbarOutside[rbar]}, drbardr[r] × DERαOutsiderbar[r]]

DERαInside[rbar_] := Module[{r = rofrbarInside[rbar]},
  dPhidrMetric[r] × alpha[r]]
```

In[∘]:=
```
DERαFull[rbar_] := Piecewise[
  {
   {DERαInside[rbar], rbar < rbarStar},
   {DERαOutside[rbar], rbar ≥ rbarStar}
  }
 ]
```

Sanity checks

In[∘]:=
```
DERαInside[rbarStar]
```

Out[∘]= 0.018042195912175805140910899390686170488987554727191464 9

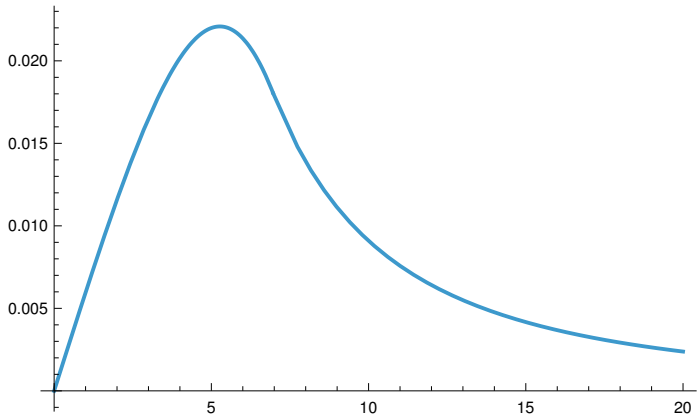
In[∘]:=
```
DERαOutside[rbarStar] // N
```

Out[∘]= 0.0180422

In[∘]:=
```
(*DERαFull[rbar_]:=If[rbar<rbarStar,DERαInside[rbar],DERαOutside[rbar]]*)
```

In[∘]:=
```
Plot[DERαFull[rbar], {rbar, 0, 20}]
```

Out[∘]=



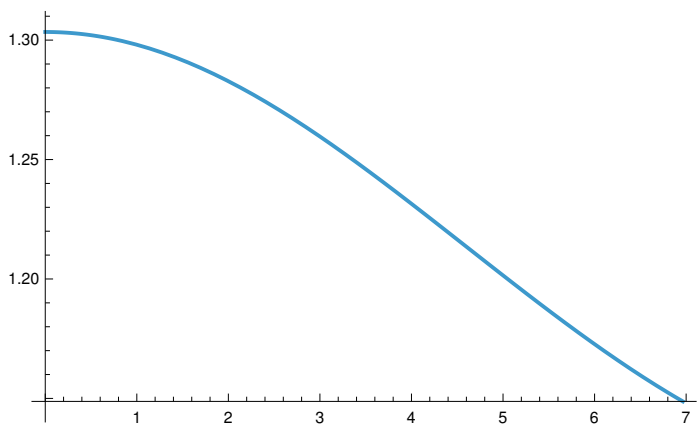
Getting AInside($\bar{r}$):

In[∘]:=
```
AInside[rbar_] := rofrbarInside[rbar] / rbar ;
```

Sanity checks

In[∘]:=
```
Plot[AInside[rbar], {rbar, 0, rbarStar}]
```

Out[∘]=



In[∘]:=
```
A[rbarStar] // N
AInside[rbarStar]
```

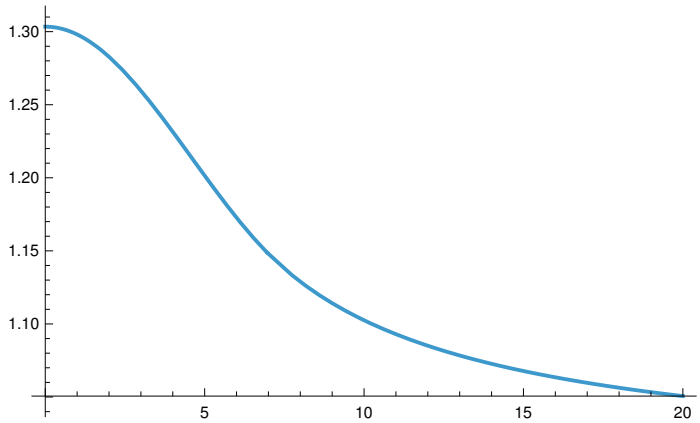
Out[∘]= 1.14875

Out[∘]= 1.1487483155918532142434341436241685156604637561356398 04

In[∘]:=
```
AFull[rbar_] := Piecewise[
   {
    {A[rbar], rbar ≥ rbarStar},
    {AInside[rbar], rbar < rbarStar}
   }
  ];
```

In[∘]:=
```
Plot[AFull[rbar], {rbar, 0, 20}]
```

Out[∘]=



Isotropic coordinates definitions of DERA:

Inside

In[∘]:=
```
drbardr[rbar_, r_] := rbar/r (1 - 2 MassFuncInside[r]/r)^(-1/2)
```

$$\text{drbardr}[\text{rbar_}, \text{r_}] := \frac{\text{rbar}}{\text{r}}\left(1 - \frac{2\,\text{MassFuncInside}[\text{r}]}{\text{r}}\right)^{-\frac{1}{2}}$$

$$\text{drdrbar}[\text{rbar_}] := \frac{\text{rofrbarInside}[\text{rbar}]}{\text{rbar}}\left(1 - \frac{2\,\text{MassFuncInside}[\text{rofrbarInside}[\text{rbar}]]}{\text{rofrbarInside}[\text{rbar}]}\right)^{\frac{1}{2}}$$

In[∘]:=
```
(*Chain rule below from AInside*)
```

In[∘]:=

$$\text{DERAInside}[\text{rbar_}] := \frac{\text{drdrbar}[\text{rbar}]}{\text{rbar}} - \frac{\text{rofrbarInside}[\text{rbar}]}{\text{rbar}^2};$$

Outside

In[∘]:=
```
DERAOutside[rbar_] := Derivative[1][A][rbar]
```

Full

In[∘]:=
```
DERAFull[rbar_] := Piecewise[
   {
    {DERAOutside[rbar], rbar ≥ rbarStar},
    {DERAInside[rbar], rbar < rbarStar}
   }
  ];
```

Sanity checks

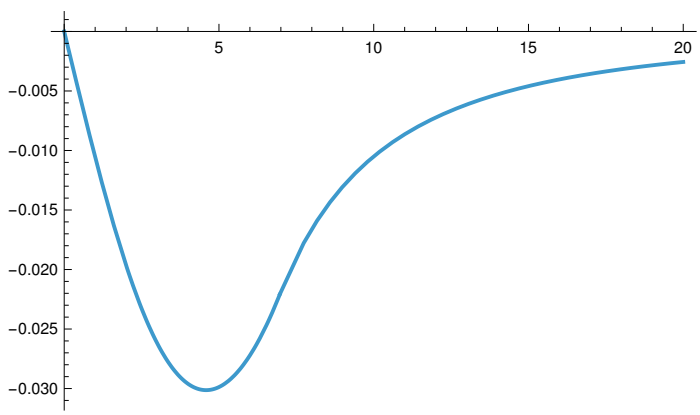
In[∘]:=
```
DERAOutside[rbarStar] // N
DERAInside[rbarStar]
```

Out[∘]= -0.0220995

Out[∘]= -0.022099489674330444219434004362804943259231258087179781

In[∘]:=
```
Plot[DERAFull[rbar], {rbar, 0, 20}]
```

Out[∘]=



Defining the speed of sound a(r):

In[∘]:=
```
cutoffIso = .999999999999999 rbarStar
```

Out[∘]= 6.9641

In[∘]:=
```
aFirst[rbar_] := Sqrt[dPdr[rofrbarInside[rbar], Pressure[rofrbarInside[rbar]]] /
   (-(2 rho0 rofrbarInside[rbar]) / RStar^2)]
```

In[∘]:=
```
aFirst[cutoffIso]
```

Out[∘]= 1.21667×10^{-8}

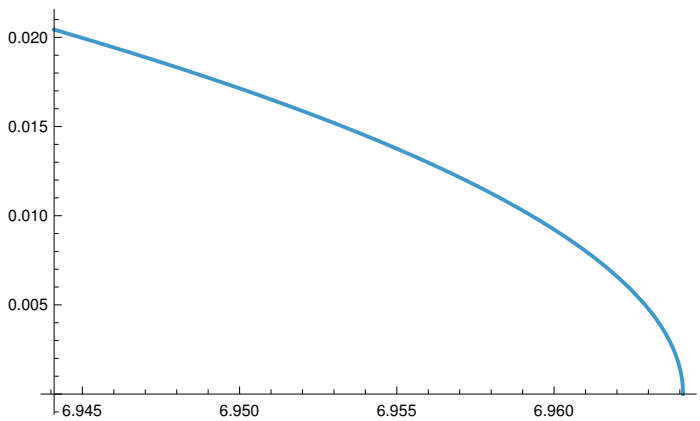
In[∘]:=
```
(*Needed to implement these measures to speed things up using
 Module to avoid recomputation and also the N@ to return numbers
 instead of symbols for NDSolve, pressure was causing an issue*)
(*Module avoids repeating the calculation three times as
 there were three arguments I would have had to have inputted it in*)
```

In[∘]:=
```
asosPiecewise[rbar_?NumericQ] := Module[{r = rofrbarInside[rbar]}, Piecewise[{
    {Sqrt[dPdr[r, Pressure[r]] / (-(2 rho0 r) / RStar^2)], rbar < cutoffIso},
    {aFirst[cutoffIso], rbar ≥ cutoffIso}
   }
  ]
 ]
```

In[∘]:=
```
(*asosPiecewise[rbar_?NumericQ]:=Module[{r=rofrbarInside[rbar]},
  N@Sqrt[dPdr[r,Pressure[r]]/(-(2 rho0 r)/RStar^2)]]*)
```

In[∘]:=
```
Plot[asosPiecewise[rbar], {rbar, cutoffIso - .02, rbarStar}]
```

Out[∘]=



Setting up 2D ODEs for new density profile:

In[∘]:=
```
Isorho[rbar_] := Module[{r = rofrbarInside[rbar]}, rhoInside[r]]
```

In[∘]:=
```
IsotropicCoordsPressure[rbar_] := Pressure[rofrbarInside[rbar]]
```

Constants that are going to be used

In[∘]:=
```
m = 10^-6; (*mass of PBH?*)
```

Mach number interpolation

Interpolating

In[∘]:=
```
fMachLow[m_] := 1/2 Log[(1 + m)/(1 - m)] - m ;
fMachhigh[m_] := 1/2 Log[1 - 1/m^2] + 10
```

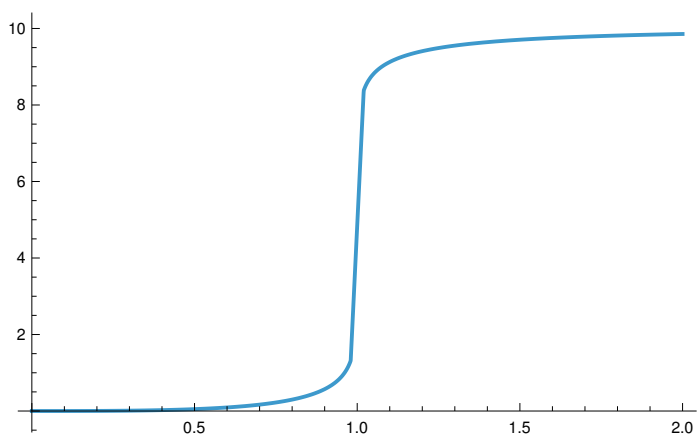
In[∘]:=
```
x1 = .98;
x2 = 1.02;
y1 = fMachLow[x1];
y2 = fMachhigh[x2];
```

In[∘]:=
```
anaylticline[Mach_] := y1 + (y2 - y1)(Mach - x1)/(x2 - x1)
```

In[∘]:=
```
IVariableInterpolated[Mach_] := Piecewise[{
    {1/2 Log[(1 + Mach)/(1 - Mach)] - Mach, Mach < 0.98},
    {anaylticline[Mach], 0.98 < Mach < 1.02},
    {1/2 Log[1 - 1/Mach^2] + 10, Mach > 1.02}
  }];
```

In[∘]:=
```
Plot[IVariableInterpolated[Mach], {Mach, 0, 2}, Exclusions → None, PlotRange → All]
```

Out[∘]=



Outside Star

Velocities

In[∘]:=
```
Ut[rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] := Sqrt[1 + (ur/A[rbar])^2 + (uphi/(rbar A[rbar]))^2] / α[rbar];

umagOutside[rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] := Sqrt[(ur^2 + (uphi/rbar)^2)] / A[rbar]
```

Rates of change of velocities

In[∘]:=
```
drBardt[rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] := 1/(A[rbar]^2 Ut[rbar, ur, uphi]) ur;

dphidtOutside[rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] :=
  1/(rbar^2 * A[rbar]^2 * Ut[rbar, ur, uphi]) uphi;
```

Gravitational Waves

In[∘]:=
```
FRRr[m_?NumericQ, rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] :=
  Module[{r = rofrbarOutside[rbar]}, ((8 m^2)/(15 r^4) * drBardt[rbar, ur, uphi] *
     (8 M^2 + r M (6 drBardt[rbar, ur, uphi]^2 + 36 r^2 dphidtOutside[rbar, ur, uphi]^2)))];
```

In[∘]:=
```
FRRphi[m_?NumericQ, rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] :=
  Module[{r = rofrbarOutside[rbar]}, ((8 m^2)/(5 r^4) * dphidtOutside[rbar, ur, uphi] *
     (2 M^2 + r M (9 drBardt[rbar, ur, uphi]^2 - 6 r^2 dphidtOutside[rbar, ur, uphi]^2)))];
```

In[∘]:=
```
duphidtOutside[m_?NumericQ, rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] :=
  η/m (A[rbar]^2 rbar^2 FRRphi[m, rbar, ur, uphi]);

durdt2DOutside[m_?NumericQ, rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] :=
  -α[rbar] × Ut[rbar, ur, uphi] × DERαOutside[rbar] + ur^2/Ut[rbar, ur, uphi] (DERAOutside[rbar]/A[rbar]^3) +
   uphi^2/Ut[rbar, ur, uphi] (1/(rbar^3 * A[rbar]^2) + DERAOutside[rbar]/(rbar^2 * A[rbar]^3)) +
   η/m (A[rbar]^2 FRRr[m, rbar, ur, uphi]);
```

Inside Star

Velocities

In[∘]:=
```
UtInside[rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] :=
```
$$\frac{\sqrt{1+\left(\frac{\text{ur}}{\text{AInside[rbar]}}\right)^2+\left(\frac{\text{uphi}}{\text{rbar AInside[rbar]}}\right)^2}}{\alpha\text{Inside[rbar]}};$$

$$\text{umagInside[rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ]} := \frac{\sqrt{\left(\text{ur}^2+\left(\frac{\text{uphi}}{\text{rbar}}\right)^2\right)}}{\text{AInside[rbar]}}$$

Mach business

In[∘]:=
```
Mach[rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] :=
```
$$\frac{\text{umagInside[rbar, ur, uphi]}}{\text{asosPiecewise[rbar]}\times\alpha\text{Inside[rbar]}\times\text{UtInside[rbar, ur, uphi]}}$$

Dissipative Forces

Accretion

In[∘]:=
```
mdot[m_?NumericQ, rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] :=
```
$$\frac{4\,\text{Pi Isorho[rbar]}\,\text{m}^2}{\left(\text{asosPiecewise[rbar]}^2+\left(\frac{\text{umagInside[rbar, ur, uphi]}}{\alpha\text{Inside[rbar]}\times\text{UtInside[rbar, ur, uphi]}}\right)^2\right)^{\frac{3}{2}}};$$

```
(*General Bondi-Hoyle-Lyttleton formula for mass accretion with q = 3 and λ=1*)
```

In[∘]:=
```
dpdtaccretionr[m_?NumericQ, rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] :=
  -((4 Pi αInside[rbar] (Isorho[rbar] + IsotropicCoordsPressure[rbar]) m^2))
   (1/((asosPiecewise[rbar]^2 + (umagInside[rbar, ur, uphi]/(αInside[rbar] × UtInside[rbar, ur, uphi]))^2)^(3/2)))
   ur;

dpdtaccretionphi[m_?NumericQ, rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] :=
  -((4 Pi αInside[rbar] (Isorho[rbar] + IsotropicCoordsPressure[rbar]) m^2))
   (1/((asosPiecewise[rbar]^2 + (umagInside[rbar, ur, uphi]/(αInside[rbar] × UtInside[rbar, ur, uphi]))^2)^(3/2)))
   uphi
```

Drag

In[∘]:=
```
dpdtdragphi[m_?NumericQ, rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] :=
  1/umagInside[rbar, ur, uphi]^3 (-4 π αInside[rbar] *
     (Isorho[rbar] + IsotropicCoordsPressure[rbar]) * m^2 * (1 + 2 umagInside[rbar, ur, uphi]^2)^2 *
     IVariableInterpolated[Mach[rbar, ur, uphi]] * uphi);

dpdtdragr[m_?NumericQ, rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] :=
  (-4 π αInside[rbar] (Isorho[rbar] + IsotropicCoordsPressure[rbar])
     m^2 (1 + 2 umagInside[rbar, ur, uphi]^2)^2
     IVariableInterpolated[Mach[rbar, ur, uphi]] * ur) / umagInside[rbar, ur, uphi]^3;
```

Rates of change of velocities

In[∘]:=
```
drBardtInside[rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] :=
  1/(AInside[rbar]^2 UtInside[rbar, ur, uphi]) ur;

dphidtInside[rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] :=
  1/(rbar^2 AInside[rbar]^2 UtInside[rbar, ur, uphi]) uphi;
```

```
In[•]:= Mp[r_] := Derivative[1][MassFuncInside][r]
        Mpp[r_] := Derivative[2][MassFuncInside][r]
        Mppp[r_] := Derivative[3][MassFuncInside][r]

In[•]:= FRRrIns[m_?NumericQ, rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] :=
          Module[{r = rofrbarInside[rbar]}, (8 m^2)/(15 r^4)*drBardtInside[rbar, ur, uphi]*
            (8 MassFuncInside[r]^2 + r MassFuncInside[r] (6 drBardtInside[rbar, ur, uphi]^2 +
                  36 r^2 dphidtInside[rbar, ur, uphi]^2 + Mp[r] - 3 r Mpp[r]) -
              r^2 (Mp[r]^2 + 3 r^2 dphidtInside[rbar, ur, uphi]^2 (6 Mp[r] - r Mpp[r]) +
                  drBardtInside[rbar, ur, uphi]^2 (2 Mp[r] + r Mpp[r] - r^2 Mppp[r])))];

In[•]:= FRRphiIns[m_?NumericQ, rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] :=
          Module[{r = rofrbarInside[rbar]}, (8 m^2)/(5 r^4)*dphidtInside[rbar, ur, uphi]*
            (2 MassFuncInside[r]^2 + 2 r^4 dphidtInside[rbar, ur, uphi]^2 × Mp[r] + r MassFuncInside[r]
                (9 drBardtInside[rbar, ur, uphi]^2 - 6 r^2 dphidtInside[rbar, ur, uphi]^2 - 2 Mp[r]) -
              r^2 drBardtInside[rbar, ur, uphi]^2 (7 Mp[r] - 2 r Mpp[r]))];

In[•]:= duphidtInside[m_?NumericQ, rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] :=
          η/m (dpdtaccretionphi[m, rbar, ur, uphi] + dpdtdragphi[m, rbar, ur, uphi] +
              AInside[rbar]^2 rbar^2 FRRphiIns[m, rbar, ur, uphi]);

        durdt2DInside[m_?NumericQ, rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] :=
          -αInside[rbar] × UtInside[rbar, ur, uphi] × DERαInside[rbar] +
            ur^2/UtInside[rbar, ur, uphi] (DERAInside[rbar]/AInside[rbar]^3) +
            uphi^2/UtInside[rbar, ur, uphi] (1/(rbar^3 AInside[rbar]^2) + DERAInside[rbar]/(rbar^2 AInside[rbar]^3)) +
            η/m (dpdtaccretionr[m, rbar, ur, uphi] + dpdtdragr[m, rbar, ur, uphi] +
              AInside[rbar]^2 FRRrIns[m, rbar, ur, uphi]);
```

Defining Energy and Angular momentum per unit mass functions along with other orbital parameters

In[∘]:=
```
EnergyPerUnitMass[ptilde_, eccen_] := Sqrt[((ptilde - 2)^2 - 4 * eccen^2)/(ptilde * (ptilde - 3 - eccen^2))];
LPerUnitMass[ptilde_, eccen_] := (ptilde * M)/Sqrt[ptilde - 3 - eccen^2];
ptildefunc[p_, Mass_] := p/Mass;
rapoapsis[eccen_?NumericQ, ptilde_?NumericQ, Mass_?NumericQ] := ((ptilde) * Mass)/(1 - eccen);
(*In Schwarzschild areal r coordinates*)
rperiapsis[eccen_, ptilde_] := ptilde/(1 + eccen);
```

Defining the constants to define the motion

Eccentricity and semi-latus rectum

In[∘]:=
```
eccentricity = 0.4;
(*eccentricity = 9/11;*)(*Obtained from setting periapsis to 5.5 and ptilde still to 10*)
semilatrecP = 10;
ptilde = ptildefunc[semilatrecP, M];
```

Energy and Angular momentum per unit mass

In[∘]:=
```
Energy = EnergyPerUnitMass[ptilde, eccentricity];
AngularMomentumUphi0 = LPerUnitMass[ptilde, eccentricity];
```

Initial drop radius (apoapsis), closest point it goes to (perihelion), defining Isotropic star radius

In[∘]:=
```
r0 = rapoapsis[eccentricity, ptilde, M];
rbar0 = rbarofrOutside[r0];

rperiapsis[eccentricity, ptilde];
(*Setting it up such that it passes the point where I know the accretion is
 strong enough to sufficiently slow it down from the inside simulation.
  (Needed this for accretion and drag to work independently and together, GW allowed
     me to go back to it not having to pass through the star on the first orbit)
 Used this to find the proper rapoapsis*)
```

In[∘]:=
```
(*r_a + r_p = 2a, a = p/(1-e^2) = 30.25
    r_a = 60.5-5.5 = 55 = rbar0 = r_apoapsis*)
```

Must satisfy this inequality to pass through star

In[∘]:=
```
ptilde < RStar (1 + eccentricity)
ptilde > 6 + 2 eccentricity
```

Out[∘]= True

Out[∘]= True

Solving the ODEs

In[∘]:=
```
drBardtInside[rbarStar, 0, AngularMomentumUphi0]
durdt2DInside[m, rbarStar, 0, AngularMomentumUphi0]
duphidtInside[m, rbarStar, 0, AngularMomentumUphi0]
dphidtInside[rbarStar, 0, AngularMomentumUphi0]
```

Out[∘]= 0.

Out[∘]= 0.00219943

Out[∘]= $-5.4786 \times 10^{-9}\,\eta$

Out[∘]= 0.0466817

In[∘]:=
```
t0 = 0;
tEnd = 50 000;
η = 10^4;

Monitor[
  solIn = NDSolve[
     {
      rbar'[t] ==
       Piecewise[
        {
```

```
      {drBardt[rbar[t], ur[t], uphi[t]], rbar[t] ≥ rbarStar},
      {drBardtInside[rbar[t], ur[t], uphi[t]], rbar[t] < rbarStar}
    }
  ],
 ur'[t] ==
  Piecewise[
   {
    {durdt2DOutside[mass[t], rbar[t], ur[t], uphi[t]], rbar[t] ≥ rbarStar},
    {durdt2DInside[mass[t], rbar[t], ur[t], uphi[t]], rbar[t] < rbarStar}
   }
  ],
 uphi'[t] ==
  Piecewise[
   {
    {duphidtOutside[mass[t], rbar[t], ur[t], uphi[t]], rbar[t] ≥ rbarStar},
    {duphidtInside[mass[t], rbar[t], ur[t], uphi[t]], rbar[t] < rbarStar}
   }
  ],
 phi'[t] ==
  Piecewise[
   {
    {dphidtOutside[rbar[t], ur[t], uphi[t]], rbar[t] ≥ rbarStar},
    {dphidtInside[rbar[t], ur[t], uphi[t]], rbar[t] < rbarStar}
   }
  ],
 mass'[t] ==
  Piecewise[
   {
    {0, rbar[t] ≥ rbarStar},
    {mdot[mass[t], rbar[t], ur[t], uphi[t]], rbar[t] < rbarStar}
   }
  ],
 rbar[t0] == rbar0,
 ur[t0] == 0,
 phi[t0] == 0,
 uphi[t0] == AngularMomentumUphi0,
 mass[t0] == m,

 WhenEvent[rbar[t] ≤ 10^-2 , "StopIntegration"]
},
{rbar, ur, phi, uphi, mass},
{t, t0, tEnd},
```

```
      MaxSteps → Infinity,
      WorkingPrecision → MachinePrecision,
      StepMonitor :→ (currentTime = t) (*Machine precision and 20k max steps also works*)
     ][[1]],
   Column[{
     Row[{"Progress: ", ProgressIndicator[currentTime, {t0, tEnd}]}],
     Row[{"Current Time: ", NumberForm[currentTime, {6, 2}], " / ", tEnd}]
     (*Amount of digits displayed, amount of decimals rounded*)
    }]
  ];
```

In[∘]:=
```
rbarNumIn = rbar /. solIn;
urNumIn = ur /. solIn;
phiNumIn = phi /. solIn;
uphiNumIn = uphi /. solIn;
massNumIn = mass /. solIn;
```

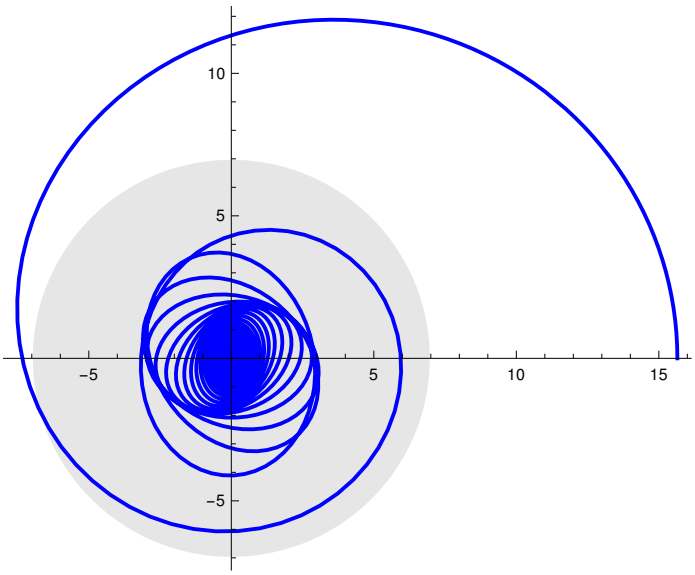
In[∘]:=
```
tFinish = rbarNumIn["Domain"][[1, 2]]
tSurface = t /. FindRoot[rbarNumIn[t] == rbarStar, {t, 10, 10, tFinish}]
```

Out[∘]= 7266.22

Out[∘]= 159.791

In[∘]:=
```
x[t_] := rbarNumIn[t] Cos[phiNumIn[t]];
y[t_] := rbarNumIn[t] Sin[phiNumIn[t]];
orbit2D = ParametricPlot[{x[t], y[t]}, {t, t0, 0.999 tFinish}, AspectRatio → Automatic,
   AxesLabel → {"x", "y"}, PlotRange → All, PlotStyle → {Thick, Blue}];
(*Automatic scale makes the aces the same scale*)
centralBody = Graphics[{GrayLevel[0.9], Disk[{0, 0}, rbarStar]}];
Show[centralBody, orbit2D, AspectRatio → Automatic, PlotRange → All, Axes → True]
```

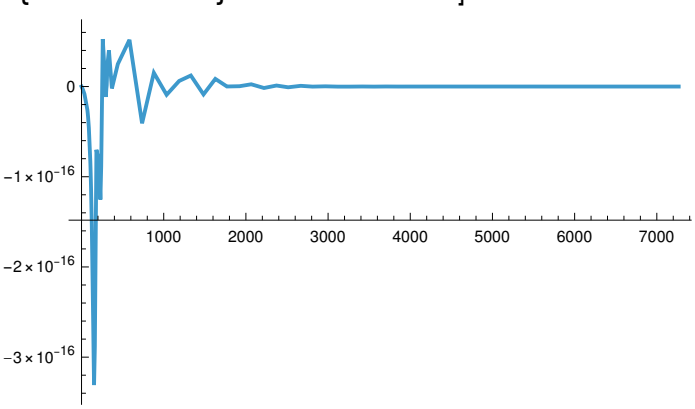
Out[∘]=



In[∘]:=
```
FRRrFull[m_, rbar_, ur_?NumericQ, uphi_?NumericQ] := Piecewise[
  {
   {FRRr[m, rbar, ur, uphi], rbar ≥ rbarStar},
   {FRRrIns[m, rbar, ur, uphi], rbar < rbarStar}
  }
 ]
```

In[∘]:=
```
Plot[FRRrFull[massNumIn[t], rbarNumIn[t], urNumIn[t], uphiNumIn[t]],
 {t, 0, tFinish}, PlotRange → All]
```

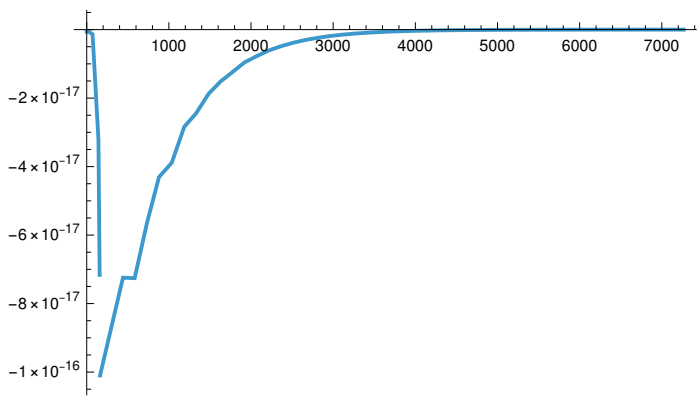
Out[∘]=



In[∘]:=
```
FRRphiFull[m_, rbar_, ur_, uphi_] := Piecewise[
  {
   {FRRphi[m, rbar, ur, uphi], rbar ≥ rbarStar},
   {FRRphiIns[m, rbar, ur, uphi], rbar < rbarStar}
  }
 ]
```

In[∘]:=
```
Plot[FRRphiFull[massNumIn[t], rbarNumIn[t], urNumIn[t], uphiNumIn[t]],
 {t, 0, tFinish}, PlotRange → All]
```

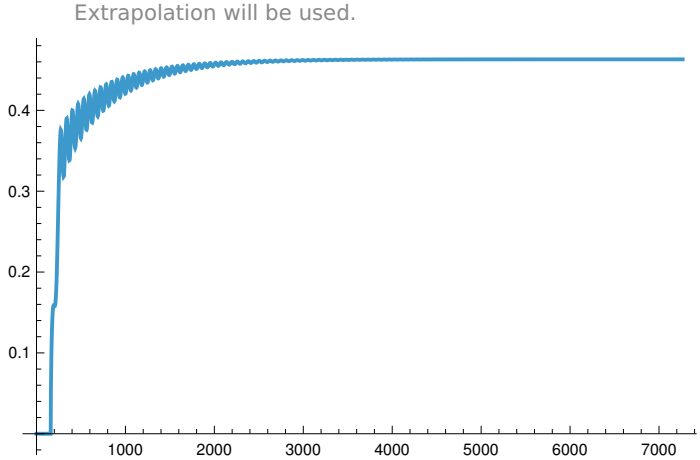
Out[∘]=



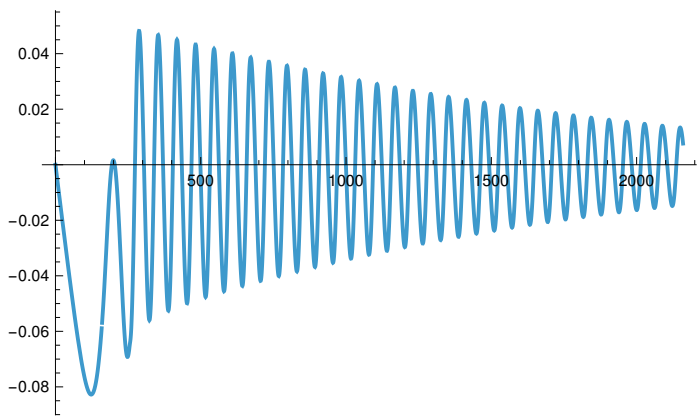
In[∘]:=
```
Plot[asosPiecewise[rbarNumIn[t]], {t, 0, tFinish}, PlotRange → All]
```

InterpolatingFunction: Input value {15.6507} lies outside the range of data in the interpolating function. Extrapolation will be used.

Out[∘]=



In[∘]:=
```
Plot[
 {
  Piecewise[
   {
    {drBardt[rbarNumIn[t], urNumIn[t], uphiNumIn[t]], rbarNumIn[t] ≥ rbarStar},
    {drBardtInside[rbarNumIn[t], urNumIn[t], uphiNumIn[t]], rbarNumIn[t] < rbarStar}
   }
  ]
 },
 {t, 0, tSurface + 2000}, PlotRange → All
]
```

Out[∘]=


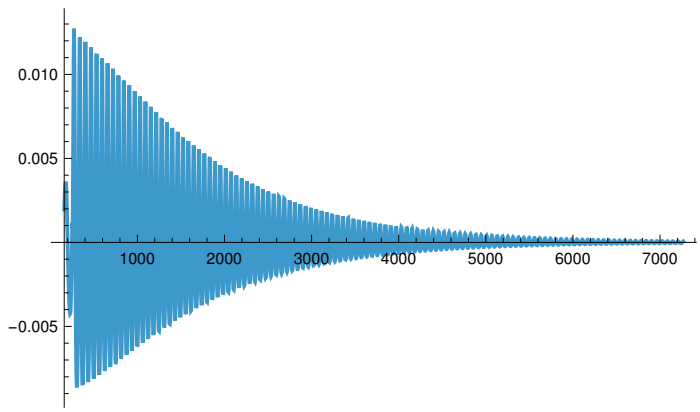
In[∘]:=
```
drBardt[rbarNumIn[tSurface], urNumIn[tSurface], uphiNumIn[tSurface]]
drBardtInside[rbarNumIn[tSurface], urNumIn[tSurface], uphiNumIn[tSurface]]
```

Out[∘]= -0.0579281

Out[∘]= -0.0579281

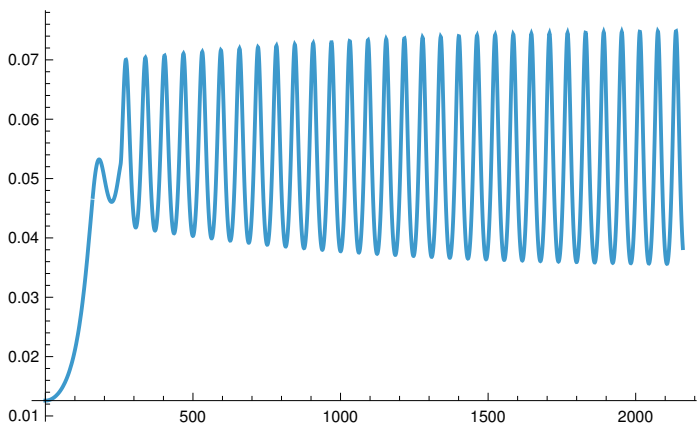
In[∘]:=
```
Plot[
 {
  Piecewise[
   {
    {durdt2DOutside[rbarNumIn[t], urNumIn[t], uphiNumIn[t]], rbarNumIn[t] ≥ rbarStar},
    {durdt2DInside[massNumIn[t],
       rbarNumIn[t], urNumIn[t], uphiNumIn[t]], rbarNumIn[t] < rbarStar}
   }
  ]
 },
 {t, 0, tFinish}, PlotRange → All
]
```

Out[∘]=

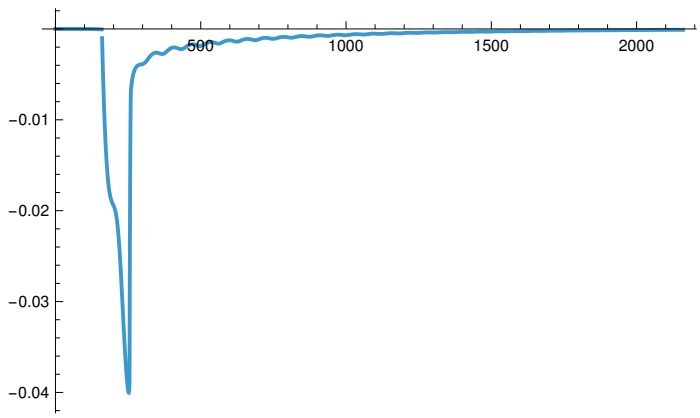


In[∘]:=
```
Plot[
 {
  Piecewise[
   {
    {dphidtOutside[rbarNumIn[t], urNumIn[t], uphiNumIn[t]], rbarNumIn[t] ≥ rbarStar},
    {dphidtInside[rbarNumIn[t], urNumIn[t], uphiNumIn[t]], rbarNumIn[t] < rbarStar}
   }
  ]
 },
 {t, 0, tSurface + 2000}, PlotRange → All
]
```

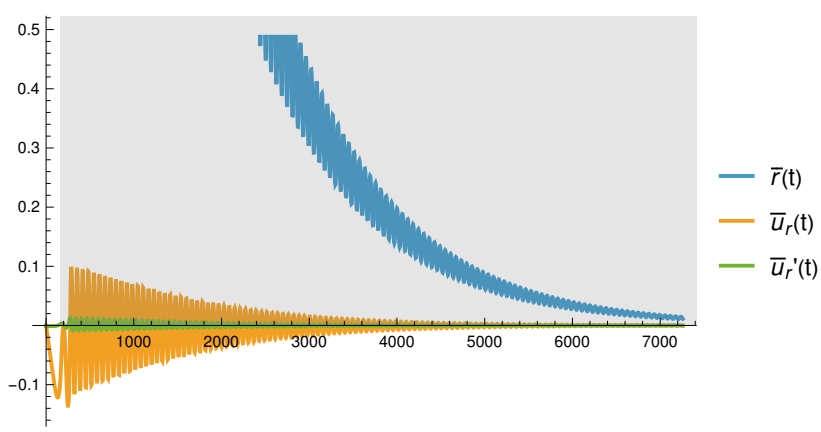
Out[∘]=



In[∘]:=
```
Plot[
 {
  Piecewise[
   {
    {duphidtOutside[massNumIn[t], rbarNumIn[t], urNumIn[t], uphiNumIn[t]],
      rbarNumIn[t] ≥ rbarStar},
    {duphidtInside[massNumIn[t],
       rbarNumIn[t], urNumIn[t], uphiNumIn[t]], rbarNumIn[t] < rbarStar}
   }
  ]
 },
 {t, 0, tSurface + 2000}, PlotRange → All
]
```

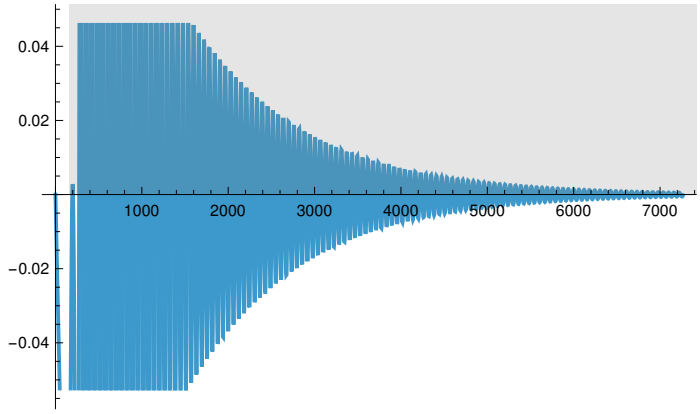
Out[∘]=


In[∘]:=
```
outFigRadial = Show[Plot[{rbarNumIn[t], urNumIn[t], urNumIn'[t]},
    {t, t0, .999 tFinish}, PlotLegends → LineLegend[Automatic, {"r̄(t)", "ū_r(t)", "ū_r'(t)"}]],
  Plot[rbarStar, {t, tSurface, tSurface + tFinish},
   PlotStyle → Directive[Gray, Opacity[0.1]], Filling → Bottom]]
```

Out[∘]=


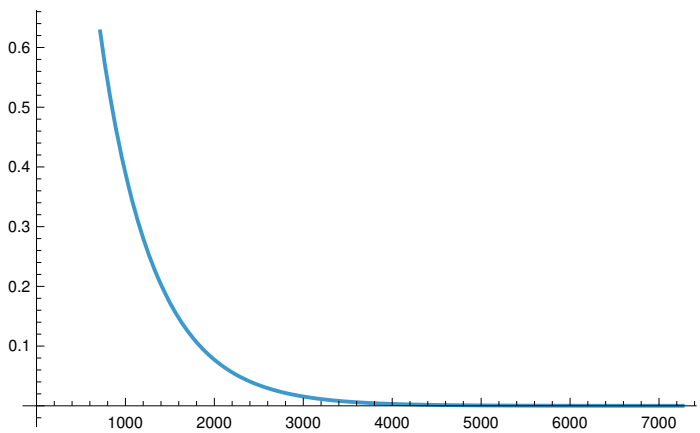
In[∘]:=
```
outFigRadial = Show[Plot[{urNumIn[t]}, {t, t0, tFinish},
    PlotLegends → LineLegend[Automatic, {"r̄(t)", "ū_r(t)", "ū_r'(t)"}]],
  Plot[rbarStar, {t, tSurface, tSurface + tFinish},
    PlotStyle → Directive[Gray, Opacity[0.1]], Filling → Bottom]]
```

Out[∘]=



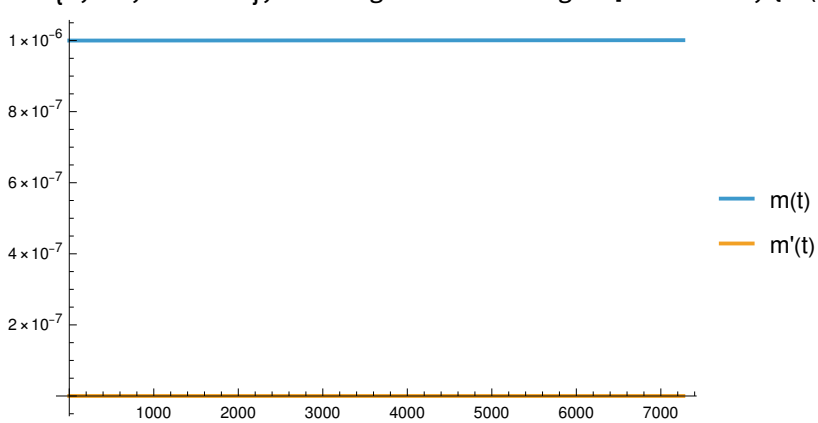
In[∘]:=
```
outFigAngular = Plot[{uphiNumIn[t]}, {t, t0, tFinish},
  PlotLegends → LineLegend[Automatic, {"phi(t)", "uphi(t)", "uphi'(t)"}]]
```

Out[∘]=



In[∘]:=
```
outFigMass = Show[Plot[{massNumIn[t], massNumIn'[t]},
    {t, t0, tFinish}, PlotLegends → LineLegend[Automatic, {"m(t)", "m'(t)"}]]]
```

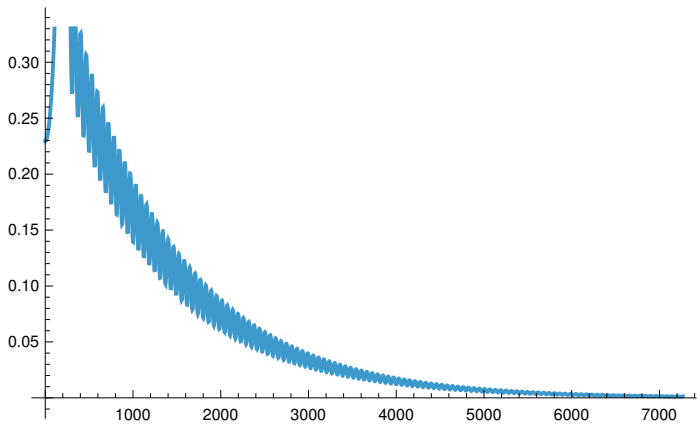
Out[∘]=



In[∘]:=
```
umag[rbar_, ur_, uphi_] := Piecewise[
  {
   {umagOutside[rbar, ur, uphi], rbar ≥ rbarStar},
   {umagInside[rbar, ur, uphi], rbar < rbarStar}
  }
 ]
```

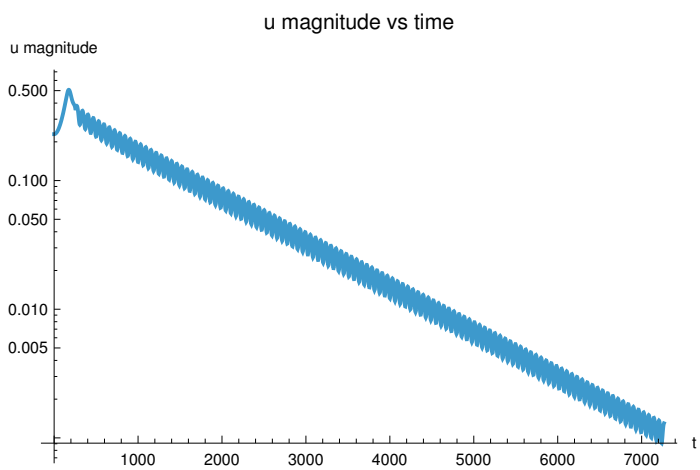
In[∘]:=
```
Plot[umag[rbarNumIn[t], urNumIn[t], uphiNumIn[t]], {t, 0, tFinish}, Exclusions → None]
(*Normal behaviour judging from rho=const simulation*)
```

Out[∘]=



In[∘]:=
```
LogPlot[Evaluate[umag[rbarNumIn[t], urNumIn[t], uphiNumIn[t]] /. solIn], {t, 0, tFinish},
 PlotRange → All, AxesLabel → {"t", "u magnitude"}, PlotLabel → "u magnitude vs time"]
```

Out[∘]=



In[∘]:=
```
(*Show me the value of u at tfinish*)
umag[rbarNumIn[tFinish], urNumIn[tFinish], uphiNumIn[tFinish]] /. solIn
```

Out[∘]= 0.0012974

Gravitational-wave strains

h_+

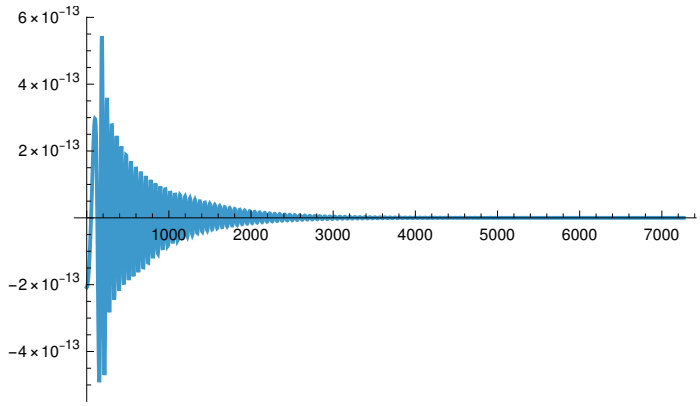
In[∘]:=
```
hplusInside[d_, m_, rbar_, ur_, uphi_, phi_] :=
 Module[{r = rofrbarInside[rbar], rd, phid},
  rd = drBardtInside[rbar, ur, uphi];
  phid = dphidtInside[rbar, ur, uphi];
  (-2 m/d)
   (MassFuncInside[r] Cos[2 phi]/r - rd^2 Cos[2 phi] + 2 rd r phid Sin[2 phi] + (r phid)^2 Cos[2 phi])
 ]
```

In[∘]:=
```
hplusOutside[d_, m_, rbar_, ur_, uphi_, phi_] :=
 Module[{r = rofrbarOutside[rbar], rd, phid},
  rd = drBardt[rbar, ur, uphi];
  phid = dphidtOutside[rbar, ur, uphi];
  (-2 m/d)(M Cos[2 phi]/r - rd^2 Cos[2 phi] + 2 rd r phid Sin[2 phi] + (r phid)^2 Cos[2 phi])
 ]
```

In[∘]:=
```
hplusFull[d_, m_, rbar_, ur_, uphi_, phi_] := Piecewise[
  {
   {hplusOutside[d, m, rbar, ur, uphi, phi], rbar ≥ rbarStar},
   {hplusInside[d, m, rbar, ur, uphi, phi], rbar < rbarStar}
  }
 ]
```

In[∘]:=
```
Plot[hplusFull[10^6, massNumIn[t], rbarNumIn[t], urNumIn[t], uphiNumIn[t], phiNumIn[t]],
 {t, 0, tFinish}, PlotRange → Full, Exclusions → None] (*10^8 it gets jagged*)
```

Out[∘]=



h_x

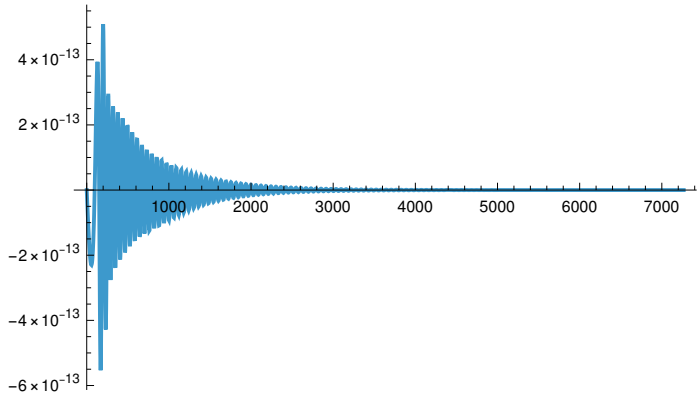
In[∘]:=
```
hxInside[d_, m_, rbar_, ur_, uphi_, phi_] := Module[{r = rofrbarInside[rbar], rd, phid},
  rd = drBardtInside[rbar, ur, uphi];
  phid = dphidtInside[rbar, ur, uphi];
  (-2 m / d)
   (MassFuncInside[r] Sin[2 phi] / r - 2 (rd Sin[phi] + r phid Cos[phi]) (rd Cos[phi] - r phid Sin[phi]))
 ]
```

In[∘]:=
```
hxOutside[d_, m_, rbar_, ur_, uphi_, phi_] := Module[{r = rofrbarOutside[rbar], rd, phid},
  rd = drBardt[rbar, ur, uphi];
  phid = dphidtOutside[rbar, ur, uphi];
  (-2 m / d) (M Sin[2 phi] / r - 2 (rd Sin[phi] + r phid Cos[phi]) (rd Cos[phi] - r phid Sin[phi]))
 ]
```

In[∘]:=
```
hxFull[d_, m_, rbar_, ur_, uphi_, phi_] := Piecewise[
  {
   {hxOutside[d, m, rbar, ur, uphi, phi], rbar ≥ rbarStar},
   {hxInside[d, m, rbar, ur, uphi, phi], rbar < rbarStar}
  }
 ]
```

In[∘]:=
```
Plot[hxFull[10^6, massNumIn[t], rbarNumIn[t], urNumIn[t], uphiNumIn[t], phiNumIn[t]],
 {t, 0, tFinish}, PlotRange → Full, Exclusions → None] (*10^8 it gets jagged*)
```

Out[∘]=



Energy loss during orbits **CHECK angles and other assumptions made by authors for these formulae especially the angular momentum stuff.**

Instantaneous energy loss (coordinate system dependent)

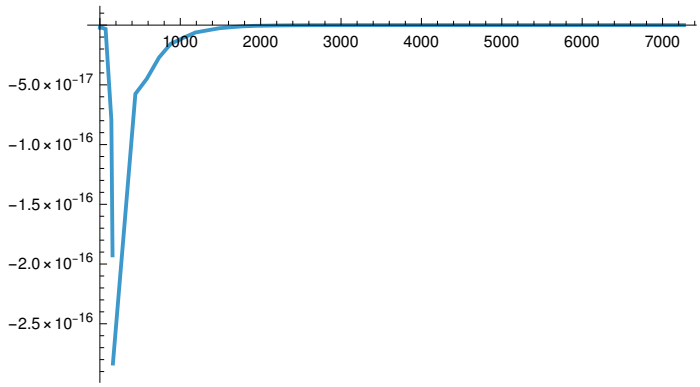
In[∘]:=
```
dEdtInside[m_, rbar_, ur_, uphi_] :=
 Module[{r = rofrbarInside[rbar], rd, phid}, rd = drBardtInside[rbar, ur, uphi];
  phid = dphidtInside[rbar, ur, uphi];
  (8 m^2)/(15 r^4)*(MassFuncInside[r]^2*(8 rd^2 + 6 (r phid)^2) + r MassFuncInside[r]*
       (6 rd^4 - 6 (3 (r phid)^4 + (r phid)^2 Mp[r]) + rd^2 (63 (r phid)^2 + Mp[r] - 3 r Mpp[r])) +
     r^2*(6 (r phid)^4 Mp[r] - rd^2 ((Mp[r])^2 + 3 (r phid)^2 (13 Mp[r] - 3 r Mpp[r])) -
        rd^4 (2 Mp[r] + r Mpp[r] - r^2 Mppp[r])))]
```

In[∘]:=
```
dEdtOutside[m_, rbar_, ur_, uphi_] :=
 Module[{r = rofrbarOutside[rbar], rd, phid}, rd = drBardt[rbar, ur, uphi];
  phid = dphidtOutside[rbar, ur, uphi];
  (8 m^2)/(15 r^4)*
   (M^2*(8 rd^2 + 6 (r phid)^2) + r M*(6 rd^4 - 6 (3 (r phid)^4) + rd^2 (63 (r phid)^2)))]
```

In[∘]:=
```
dEdtFull[m_, rbar_, ur_, uphi_] := Piecewise[
  {
   {dEdtOutside[m, rbar, ur, uphi], rbar ≥ rbarStar},
   {dEdtInside[m, rbar, ur, uphi], rbar < rbarStar}
  }
 ]
```

In[∘]:=
```
Plot[dEdtFull[massNumIn[t], rbarNumIn[t], urNumIn[t], uphiNumIn[t]],
 {t, 0, tFinish}, PlotRange → All]
```

InterpolatingFunction: Input value {15.6507} lies outside the range of data in the interpolating function. Extrapolation will be used.

Out[∘]=


Average, physical energy loss per orbit N/A

In[∘]:=
```
dEdtAvg[m_, M_, a_, e_] := -(32 m^2 M^3) / (5 a^5 (1 - e^2)^(7/2)) * (1 + (73/24) e^2 + (37/96) e^4)
```

Need formulae for M[t]? Although accretion is very small. a(t) and e(t)

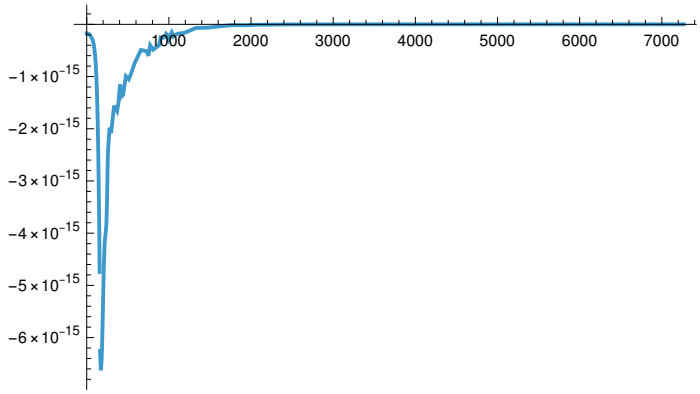
Angular momentum loss during orbits

Instantaneous angular momentum loss (coordinate system dependent)

In[∘]:=
```
dLdtOutside[m_, rbar_, ur_, uphi_] := Module[{r = rofrbarOutside[rbar]},
  r^2 FRRphi[m, rbar, ur, uphi]]
dLdtInside[m_, rbar_, ur_, uphi_] := Module[{r = rofrbarInside[rbar]},
  r^2 FRRphiIns[m, rbar, ur, uphi]]
```

In[∘]:=
```
dLdtFull[m_, rbar_, ur_, uphi_] := Piecewise[
  {
   {dLdtOutside[m, rbar, ur, uphi], rbar ≥ rbarStar},
   {dLdtInside[m, rbar, ur, uphi], rbar < rbarStar}
  }
 ]
```

In[∘]:=
```
Plot[dLdtFull[massNumIn[t], rbarNumIn[t], urNumIn[t], uphiNumIn[t]],
 {t, 0, tFinish}, PlotRange → All]
```

Out[∘]=


Average, physical angular momentum loss per orbit N/A

In[∘]:=
```
(*Formula might need to be adjusted for me as they use the angle phi =
 0 for when it crosses the star surface which is not true for me*)
```

In[∘]:=
```
dLdtAvg[m_, M_, a_, e_] := -(32 m ^ 2 M ^ (5 / 2)) / (5 a ^ (7 / 2) (1 - e ^ 2) ^ (2)) * (1 + (7 / 8) e ^ 2)
```

Need formulae for M[t]? Although accretion is very small. a(t) and e(t).

Use E and L to find a(t) and e(t)??

Capturing the PBH from infinity

Assuming PBH comes from infinity and just hits in and enters the star

Approaching the problem differently, using Keplerian orbits on the outside of the star and then GR on the inside to reduce the stiffness of the problem causing the error in the solver.

Setting v_inf to something physical is essential, most dark matter comes from the galactic halo, where things move at roughly 200km/s so with setting c = 1, vinf should be $6.67\,x\,10^{-4}$ which implies the Energy at infinity should be approximately equal to 1 and very little energy loss is needed to bound the orbit.

Defining Initial Data Functions

In[∘]:=
```
vinf = 6.67 × 10^-4;
```

In[∘]:=
```
(*rbar0 = 15.67;*)
```

In[∘]:=
```
InitialEnergyAtInfinity[vinf_] := 1/Sqrt[1 - vinf^2]; (*eq47 2404 paper*)
```

In[∘]:=
```
(*general formula for the magnitude of the velocity,
for a general spherically symmetric Isotropic metric*)
umagVel[ur_, uphi_, rbar_] := Sqrt[ur^2 + (uphi/rbar)^2]
umagE[Energy_, rbar_] := A[rbar] Sqrt[(Energy/α[rbar])^2 - 1]; (*eq 48 in paper*)
```

In[∘]:=
```
(*Defining the Impact angle,
which is the angle between the radial vector and the PBH's velocity vector*)
(*It is the angle the PBH enters the star*)

urInit[psi_, Einit_] := -umagE[Einit , rbarStar] Cos[psi];
uphiInit[psi_, rbarStar_, Einit_] := rbarStar umagE[Einit] Sin[psi];
(*Is defining angular momentum, L is equal to this*)
```

Setting the Initial conditions

In[∘]:=
```
(*we start at rbarStar*)
phi0 = 0;
Einf = InitialEnergyAtInfinity[vinf] // N
 u = umagE[Einf, rbarStar];
ImpactAngle = .85 Pi / 2; (*In radians,
measured from center of star to the position of particle,
in unit circle fashion, (*1.5 Pi/4  η = 10^3 is nice*)
pi/2 corresponds to grazing the star as it enters and 0 corresponds to a radial drop*)
ur0 = -u * Cos[ImpactAngle];
AngularMomentumUphi0 = (rbarStar) (u) Sin[ImpactAngle];
```

Out[∘]= 1.

Sanity Check to see if the two energy formulas match up

In[∘]:=
```
Einf = InitialEnergyAtInfinity[vinf] // N
α[rbarStar]^2 Ut[rbarStar, ur0, AngularMomentumUphi0] // N (*This Ut was upper indexed*)
```

Out[∘]= 1.

Out[∘]= 1.

Performing the integration

In[∘]:=
```
(*,
Method→{"DiscontinuityProcessing"→False}*)
```

In[∘]:=
```
t0 = 0;
tEnd = 100 000;
```

```
η = 1 × 10^4;

solIn = NDSolve[
    {
     rbar'[t] ==
      Piecewise[
       {
        {drBardt[rbar[t], ur[t], uphi[t]], rbar[t] ≥ rbarStar},
        {drBardtInside[rbar[t], ur[t], uphi[t]], rbar[t] < rbarStar}
       }
      ],
     ur'[t] ==
      Piecewise[
       {
        {durdt2DOutside[mass[t], rbar[t], ur[t], uphi[t]], rbar[t] ≥ rbarStar},
        {durdt2DInside[mass[t], rbar[t], ur[t], uphi[t]], rbar[t] < rbarStar}
       }
      ],
     uphi'[t] ==
      Piecewise[
       {
        {duphidtOutside[mass[t], rbar[t], ur[t], uphi[t]], rbar[t] ≥ rbarStar},
        {duphidtInside[mass[t], rbar[t], ur[t], uphi[t]], rbar[t] < rbarStar}
       }
      ],
     phi'[t] ==
      Piecewise[
       {
        {dphidtOutside[rbar[t], ur[t], uphi[t]], rbar[t] ≥ rbarStar},
        {dphidtInside[rbar[t], ur[t], uphi[t]], rbar[t] < rbarStar}
       }
      ],
     mass'[t] ==
      Piecewise[
       {
        {0, rbar[t] ≥ rbarStar},
        {mdot[mass[t], rbar[t], ur[t], uphi[t]], rbar[t] < rbarStar}
       }
      ],
     rbar[t0] == .9999 rbarStar,
     ur[t0] == ur0,
     phi[t0] == phi0,
```

```
      uphi[t0] == AngularMomentumUphi0,
      mass[t0] == m,

      WhenEvent[rbar[t] ≤ 10^-2 , "StopIntegration"]
    },
    {rbar, ur, phi, uphi, mass},
    {t, t0, tEnd},
    MaxSteps → Infinity,
    WorkingPrecision → MachinePrecision,
    Method → {"DiscontinuityProcessing" → False}
    (*Machine precision and 20k max steps also works*)
   ][[1]];
```

In[∘]:=
```
rbarNumIn = rbar /. solIn;
urNumIn = ur /. solIn;
phiNumIn = phi /. solIn;
uphiNumIn = uphi /. solIn;
massNumIn = mass /. solIn;
```

In[∘]:=
```
Abs[.9999 rbarStar - rbarStar]
```

Out[∘]= 0.00069641

In[∘]:=
```
tFinish = rbarNumIn["Domain"][[1, 2]]
tSurface = t /. FindRoot[rbarNumIn[t] == rbarStar, {t, 10, 10, tFinish}]
```

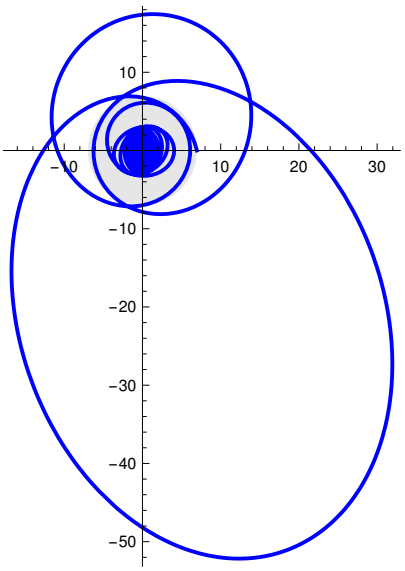
Out[∘]= 8817.02

FindRoot: The point {10.} is at the edge of the search region {10., 8817.02} in coordinate 1 and the computed search direction points outside the region.

Out[∘]= 10.

In[∘]:=
```
x[t_] := rbarNumIn[t] Cos[phiNumIn[t]];
y[t_] := rbarNumIn[t] Sin[phiNumIn[t]];
orbit2D = ParametricPlot[{x[t], y[t]}, {t, t0, 0.999 tFinish}, AspectRatio → Automatic,
   AxesLabel → {"x", "y"}, PlotRange → All, PlotStyle → {Thick, Blue}];
(*Automatic scale makes the aces the same scale*)
centralBody = Graphics[{GrayLevel[0.9], Disk[{0, 0}, rbarStar]}];
Show[centralBody, orbit2D, AspectRatio → Automatic, PlotRange → All, Axes → True]
```

Out[∘]=

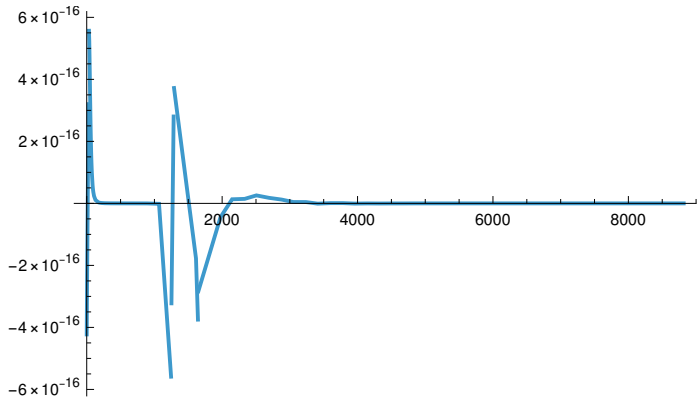


In[∘]:=
```
Animate[Show[centralBody, ParametricPlot[{x[s], y[s]}, {s, 0, t}],
   PlotRange → All, Axes → True], {t, 0, tFinish}];
```

Stuff

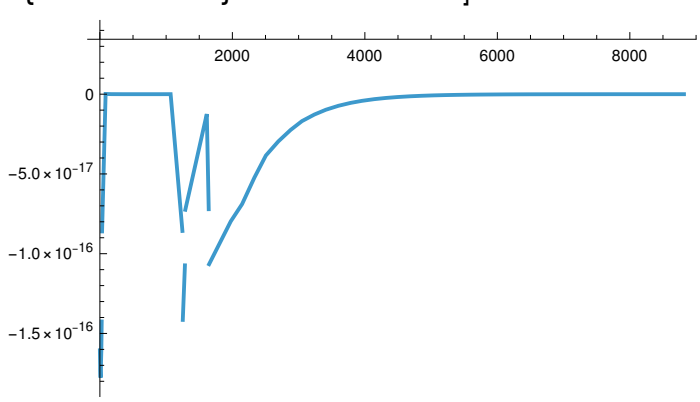
In[∘]:=
```
FRRrFull[m_, rbar_, ur_, uphi_] := Piecewise[
  {
   {FRRr[m, rbar, ur, uphi], rbar ≥ rbarStar},
   {FRRrIns[m, rbar, ur, uphi], rbar < rbarStar}
  }
 ]
```

In[∘]:=
```
Plot[FRRrFull[massNumIn[t], rbarNumIn[t], urNumIn[t], uphiNumIn[t]],
 {t, 0, tFinish}, PlotRange → All]
```

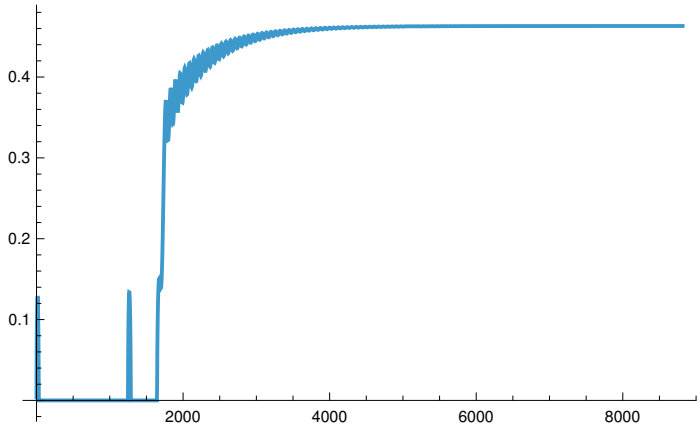
Out[∘]=


In[∘]:=
```
FRRphiFull[m_, rbar_, ur_, uphi_] := Piecewise[
  {
   {FRRphi[m, rbar, ur, uphi], rbar ≥ rbarStar},
   {FRRphiIns[m, rbar, ur, uphi], rbar < rbarStar}
  }
 ]
```

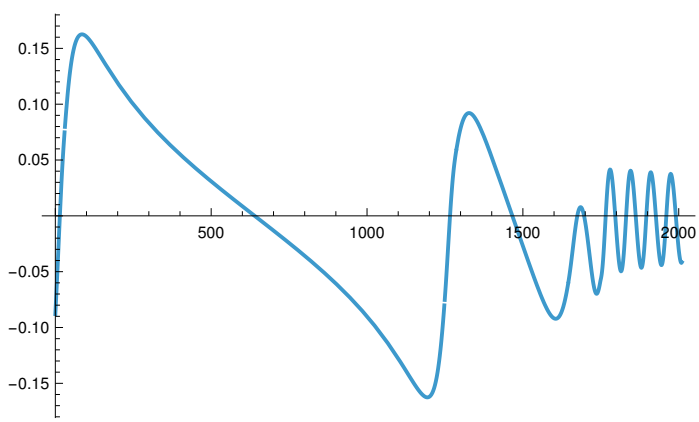
In[∘]:=
```
Plot[FRRphiFull[massNumIn[t], rbarNumIn[t], urNumIn[t], uphiNumIn[t]],
 {t, 0, tFinish}, PlotRange → All]
```

Out[∘]=


In[∘]:=
```
Plot[asosPiecewise[rbarNumIn[t]], {t, 0, tFinish}, PlotRange → All]
```

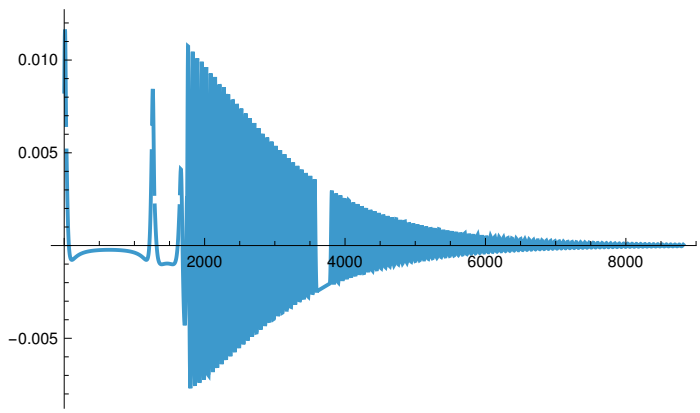
Out[∘]=


In[∘]:=
```
Plot[
 {
  Piecewise[
   {
    {drBardt[rbarNumIn[t], urNumIn[t], uphiNumIn[t]], rbarNumIn[t] ≥ rbarStar},
    {drBardtInside[rbarNumIn[t], urNumIn[t], uphiNumIn[t]], rbarNumIn[t] < rbarStar}
   }
  ]
 },
 {t, 0, tSurface + 2000}, PlotRange → All
]
```

Out[∘]=


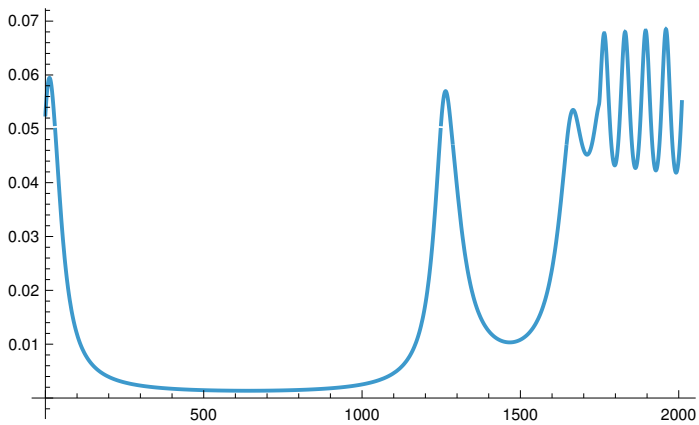
In[∘]:=
```
Plot[
 {
  Piecewise[
   {
    {durdt2DOutside[massNumIn[t], rbarNumIn[t], urNumIn[t], uphiNumIn[t]],
     rbarNumIn[t] ≥ rbarStar},
    {durdt2DInside[massNumIn[t],
       rbarNumIn[t], urNumIn[t], uphiNumIn[t]], rbarNumIn[t] < rbarStar}
   }
  ]
 },
 {t, 0, tFinish}, PlotRange → All
]
```

Out[∘]=

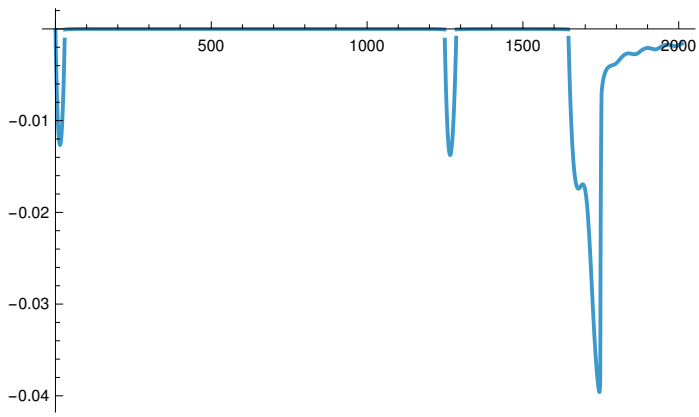


In[∘]:=
```
Plot[
 {
  Piecewise[
   {
    {dphidtOutside[rbarNumIn[t], urNumIn[t], uphiNumIn[t]], rbarNumIn[t] ≥ rbarStar},
    {dphidtInside[rbarNumIn[t], urNumIn[t], uphiNumIn[t]], rbarNumIn[t] < rbarStar}
   }
  ]
 },
 {t, 0, tSurface + 2000}, PlotRange → All
]
```

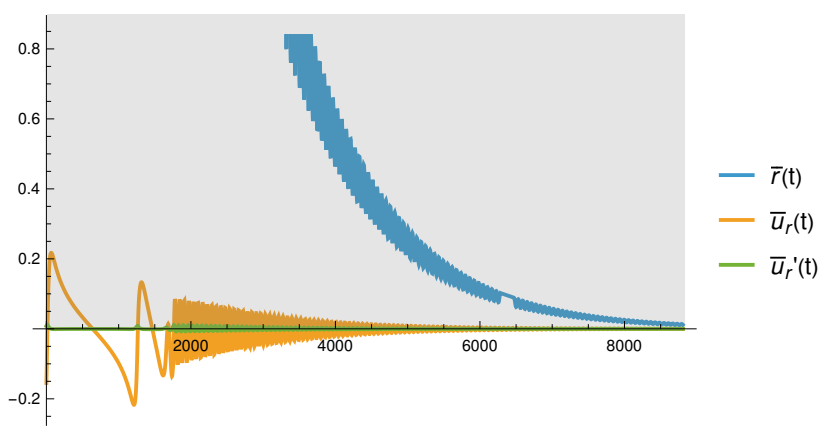
Out[∘]=



In[∘]:=
```
Plot[
 {
  Piecewise[
   {
    {duphidtOutside[massNumIn[t], rbarNumIn[t], urNumIn[t], uphiNumIn[t]],
     rbarNumIn[t] ≥ rbarStar},
    {duphidtInside[massNumIn[t],
      rbarNumIn[t], urNumIn[t], uphiNumIn[t]], rbarNumIn[t] < rbarStar}
   }
  ]
 },
 {t, 0, tSurface + 2000}, PlotRange → All
]
```

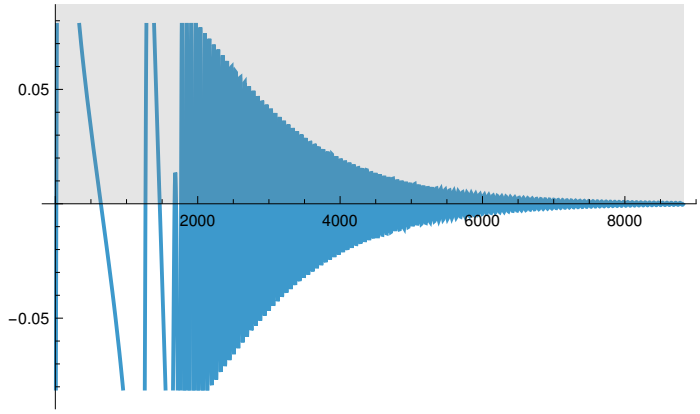
Out[∘]=


In[∘]:=
```
outFigRadial = Show[Plot[{rbarNumIn[t], urNumIn[t], urNumIn'[t]},
   {t, t0, .999 tFinish}, PlotLegends → LineLegend[Automatic, {"r̄(t)", "ū_r(t)", "ū_r'(t)"}]],
  Plot[rbarStar, {t, tSurface, tSurface + tFinish},
   PlotStyle → Directive[Gray, Opacity[0.1]], Filling → Bottom]]
```

Out[∘]=


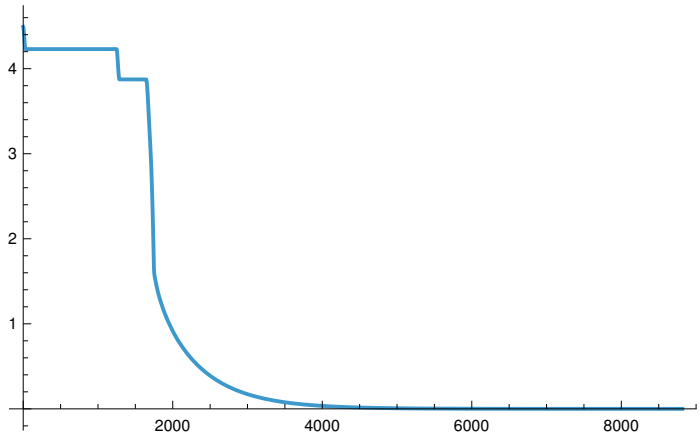
In[∘]:=
```
outFigRadial = Show[Plot[{urNumIn[t]}, {t, t0, tFinish},
    PlotLegends → LineLegend[Automatic, {"r̄(t)", "ūr(t)", "ūr'(t)"}]],
  Plot[rbarStar, {t, tSurface, tSurface + tFinish},
    PlotStyle → Directive[Gray, Opacity[0.1]], Filling → Bottom]]
```

Out[∘]=



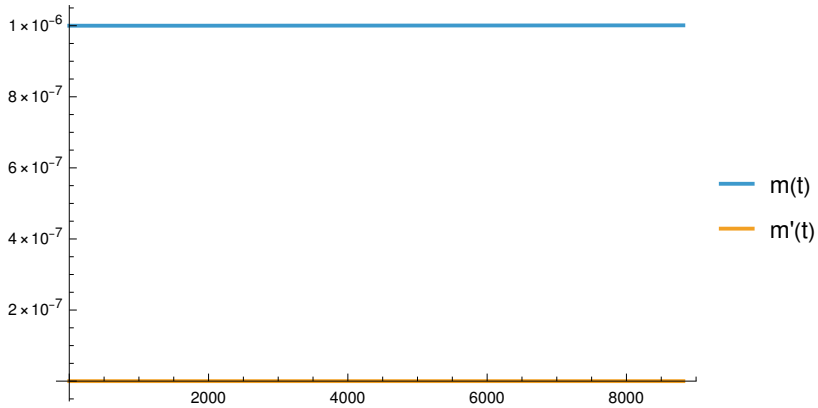
In[∘]:=
```
outFigAngular = Plot[{uphiNumIn[t]}, {t, t0, tFinish},
  PlotLegends → LineLegend[Automatic, {"phi(t)", "uphi(t)", "uphi'(t)"}]]
```

Out[∘]=



In[∘]:=
```
outFigMass = Show[Plot[{massNumIn[t], massNumIn'[t]},
   {t, t0, tFinish}, PlotLegends → LineLegend[Automatic, {"m(t)", "m'(t)"}]]]
```

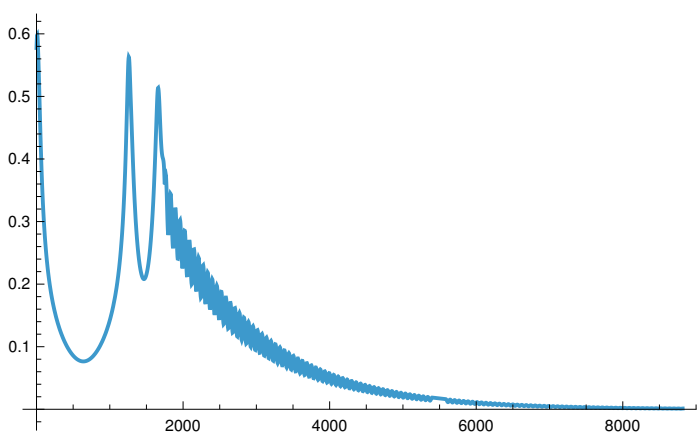
Out[∘]=



In[∘]:=
```
umag[rbar_, ur_, uphi_] := Piecewise[
  {
   {umagOutside[rbar, ur, uphi], rbar ≥ rbarStar},
   {umagInside[rbar, ur, uphi], rbar < rbarStar}
  }
 ]
```

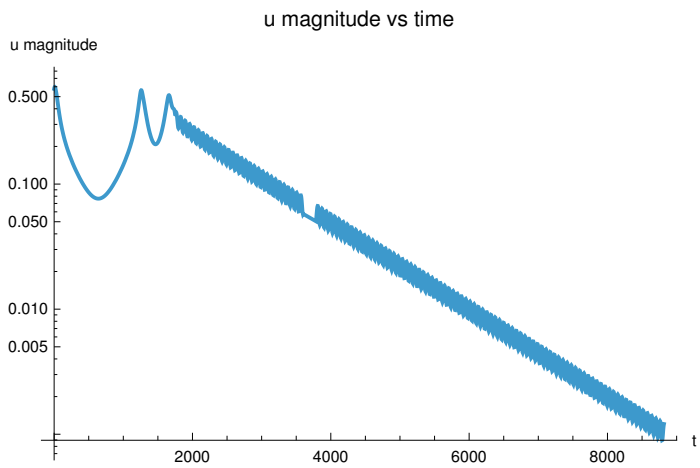
In[∘]:=
```
Plot[umag[rbarNumIn[t], urNumIn[t], uphiNumIn[t]], {t, 0, tFinish}, Exclusions → None]
(*Normal behaviour judging from rho=const simulation*)
```

Out[∘]=



In[∘]:=
```
LogPlot[Evaluate[umag[rbarNumIn[t], urNumIn[t], uphiNumIn[t]] /. solIn], {t, 0, tFinish},
 PlotRange → All, AxesLabel → {"t", "u magnitude"}, PlotLabel → "u magnitude vs time"]
```

Out[∘]=



In[∘]:=
```
(*Show me the value of u at tfinish*)
umag[rbarNumIn[tFinish], urNumIn[tFinish], uphiNumIn[tFinish]] /. solIn
```

Out[∘]= 0.001206

Gravitational-wave strains

h_+

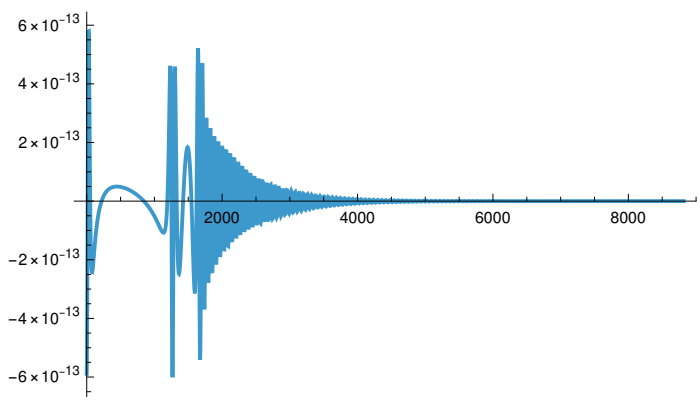
In[∘]:=
```
hplusInside[d_, m_, rbar_, ur_, uphi_, phi_] :=
 Module[{r = rofrbarInside[rbar], rd, phid},
  rd = drBardtInside[rbar, ur, uphi];
  phid = dphidtInside[rbar, ur, uphi];
  (-2 m / d)
   (MassFuncInside[r] Cos[2 phi] / r - rd^2 Cos[2 phi] + 2 rd r phid Sin[2 phi] + (r phid)^2 Cos[2 phi])
 ]
```

In[∘]:=
```
hplusOutside[d_, m_, rbar_, ur_, uphi_, phi_] :=
 Module[{r = rofrbarOutside[rbar], rd, phid},
  rd = drBardt[rbar, ur, uphi];
  phid = dphidtOutside[rbar, ur, uphi];
  (-2 m/d) (M Cos[2 phi]/r - rd^2 Cos[2 phi] + 2 rd r phid Sin[2 phi] + (r phid)^2 Cos[2 phi])
 ]
```

In[∘]:=
```
hplusFull[d_, m_, rbar_, ur_, uphi_, phi_] := Piecewise[
  {
   {hplusOutside[d, m, rbar, ur, uphi, phi], rbar ≥ rbarStar},
   {hplusInside[d, m, rbar, ur, uphi, phi], rbar < rbarStar}
  }
 ]
```

In[∘]:=
```
Plot[hplusFull[10^6, massNumIn[t], rbarNumIn[t], urNumIn[t], uphiNumIn[t], phiNumIn[t]],
 {t, 0, tFinish}, PlotRange → Full, Exclusions → None](*10^8 it gets jagged*)
```

Out[∘]=

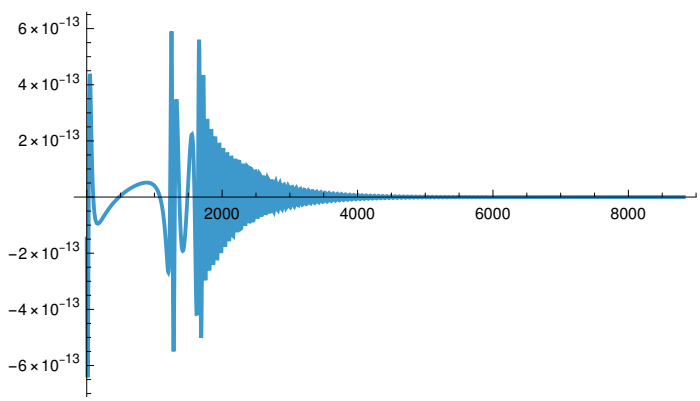


h_x

In[∘]:=
```
hxInside[d_, m_, rbar_, ur_, uphi_, phi_] := Module[{r = rofrbarInside[rbar], rd, phid},
  rd = drBardtInside[rbar, ur, uphi];
  phid = dphidtInside[rbar, ur, uphi];
  (-2 m/d) (MassFuncInside[r] Sin[2 phi]/r - 2 (rd Sin[phi] + r phid Cos[phi]) (rd Cos[phi] - r phid Sin[phi]))
 ]
```

In[∘]:=
```
hxOutside[d_, m_, rbar_, ur_, uphi_, phi_] := Module[{r = rofrbarOutside[rbar], rd, phid},
  rd = drBardt[rbar, ur, uphi];
  phid = dphidtOutside[rbar, ur, uphi];
  (-2 m/d) (M Sin[2 phi]/r - 2 (rd Sin[phi] + r phid Cos[phi]) (rd Cos[phi] - r phid Sin[phi]))
 ]
```

In[∘]:=
```
hxFull[d_, m_, rbar_, ur_, uphi_, phi_] := Piecewise[
  {
   {hxOutside[d, m, rbar, ur, uphi, phi], rbar ≥ rbarStar},
   {hxInside[d, m, rbar, ur, uphi, phi], rbar < rbarStar}
  }
 ]
```

In[∘]:=
```
Plot[hxFull[10^6, massNumIn[t], rbarNumIn[t], urNumIn[t], uphiNumIn[t], phiNumIn[t]],
 {t, 0, tFinish}, PlotRange → Full, Exclusions → None] (*10^8 it gets jagged*)
```

Out[∘]=